React Conditional Rendering — Practice Questions

Total Questions: 15
Difficulty: 3 Easy · 7 Medium · 5 Hard
Instructions: Create a separate React component for each question. Use JSX and conditional rendering where appropriate. Unless a question says otherwise, use the given sample data and do not use external libraries.

Part A — Easy (3 Questions)

Q1. Pass or Fail Result

Create a component named `Result` that stores a student's marks in a variable.

Requirements:

- If marks are **33 or above**, display `You Passed!`.
- Otherwise, display `You Failed!`.
- Test your component with marks of `72` and `21`.

Concepts: Ternary operator, comparison operators.

Q2. Login / Logout Button

Create a component named `LoginStatus` with a Boolean variable called `isLoggedIn`.

Requirements:

- If `isLoggedIn` is `true`, display a `Logout` button.
- If `isLoggedIn` is `false`, display a `Login` button.
- Also display `Welcome Back!` only when the user is logged in.

Concepts: Ternary operator, logical AND (`&&`).

Q3. Show a Notification

Create a component named `Notification` with a variable called `hasNotification`.

Requirements:

- Display the heading `Dashboard`.
- If `hasNotification` is `true`, display `You have a new notification!`.
- If it is `false`, do not display the notification message.

Concepts: Conditional rendering with `&&`.

Part B — Medium (7 Questions)

Q4. Grade Calculator

Create a component named `GradeCalculator` that stores a student's marks.

Display the grade using these rules:

Marks	Grade
90–100	A+
75–89	A
60–74	B
33–59	C
0–32	Fail

Requirements:

- Display the grade using conditional rendering.
- Display `Invalid marks` if marks are below `0` or above `100`.
- Make sure the conditions are ordered correctly.

Concepts: Multiple conditions, nested ternary or `if-else if`.

Q5. Admin and User Navigation

Create a component named `Navigation` with a Boolean variable `isAdmin`.

Requirements:

- Both roles must see the same company logo.
- Admin navigation must contain: `Dashboard`, `Manage Users`, `Reports`, `Tickets`.
- User navigation must contain: `Home`, `About Us`, `Contact Us`, `Blog`.
- Avoid writing two complete copies of the same navigation structure.
- Use an array and `map()` to render the menu items.

Concepts: Conditional data selection, reusable JSX, `map()`.

Q6. Product Availability

Create a component named `ProductCard` with these sample values:

```
const product = {
  name: "Wireless Headphones",
  price: 2499,
  inStock: false,
  discount: 15
};
```

Requirements:

- Always display the product name and price.

- If `inStock` is `true`, show an `Add to Cart` button.
- Otherwise, show `Out of Stock`.
- Show a `15% OFF` badge only when the discount is greater than `0`.

Concepts: Multiple independent conditions, `&&`, ternary operator.

Q7. Loading, Error, and Success

Create a component named `DataPage` with these variables:

```
const isLoading = false;
const hasError = false;
const users = ["Aarav", "Diya", "Kabir"];
```

Requirements:

- If loading, display `Loading users...`.
- If there is an error, display `Unable to load users.`.
- If loading is finished, there is no error, and users exist, display their names in a list.
- If loading is finished, there is no error, and the list is empty, display `No users found.`.
- Ensure that only the appropriate state is displayed at a time.

Concepts: Early returns, mutually exclusive states, arrays.

Q8. Order Status Message

Create a component named `OrderStatus` with a variable `status`.

Possible values: `"pending"`, `"shipped"`, `"delivered"`, `"cancelled"`.

Requirements:

- Show a suitable message for each status.
- Display `Unknown order status` for any other value.
- Solve it using a `switch` statement or a separate function that returns the appropriate message.

Concepts: `switch`, default cases, conditional output.

Q9. Student Attendance List

Use the following data:

```
const students = [
  { id: 1, name: "Aarav", present: true },
  { id: 2, name: "Diya", present: false },
  { id: 3, name: "Kabir", present: true },
  { id: 4, name: "Meera", present: false }
];
```

Requirements:

- Display all student names.
- Display `Present` or `Absent` beside each name using conditional rendering.
- Display `Perfect Attendance` only if every student is present.
- Display `No students available` if the array is empty.
- Use each student's `id` as the React key.

Concepts: `map()`, ternary operator, `every()`, empty-state rendering.

Q10. Show or Hide Password Field

Create a component named `PasswordField` with a Boolean variable `showPassword`.

Requirements:

- When `showPassword` is `true`, render an input with `type="text"`.
- When `showPassword` is `false`, render an input with `type="password"`.
- Display `Password is visible` only when the password is visible.
- For this question, you may change the variable manually to test both states. Do not use `useState`.

Concepts: Conditional attributes, ternary operator, conditional messages.

Part C — Hard (5 Questions)

Q11. Role-Based Dashboard

Create a component named `Dashboard` that supports three roles: `"admin"`, `"teacher"`, and `"student"`.

Requirements:

- **Admin:** show `Manage Users`, `View Reports`, and `System Settings`.
- **Teacher:** show `My Classes`, `Attendance`, and `Enter Marks`.
- **Student:** show `My Courses`, `Assignments`, and `My Results`.
- For any unknown role, show `Access Denied`.
- Display the role name in the page heading.
- Reuse the same overall layout and render menu items from an array. Do not duplicate the whole JSX for every role.

Concepts: Multiple roles, configuration objects, conditional rendering with `map()`.

Q12. Shopping Cart Summary

Use this sample data:

```
const cartItems = [
  { id: 1, name: "Keyboard", price: 1200, quantity: 1 },
  { id: 2, name: "Mouse", price: 600, quantity: 2 },
```

```
  { id: 3, name: "USB Cable", price: 200, quantity: 0 }
];
```

Requirements:

- Only display items whose quantity is greater than `0`.
- If no purchasable items remain, display `Your cart is empty`.
- Otherwise, display each item's name, quantity, and line total (`price * quantity`).
- Display `Free Delivery` when the total is at least `₹2000`; otherwise display `Delivery charge: ₹100`.
- Display the final total, including delivery when applicable.
- Do not hardcode the final total.

Concepts: `filter()`, `map()`, `reduce()`, conditional rendering, derived values.

Q13. Quiz Feedback and Results

Create a component named `QuizQuestion` using this sample data:

```
const question = {
  text: "Which keyword declares a block-scoped variable?",
  options: ["var", "let", "print", "echo"],
  correctAnswer: "let"
};

const selectedAnswer = "var";
const isSubmitted = true;
const isLastQuestion = false;
```

Requirements:

- Display the question and all options.
- Before submission, do not display any correctness feedback.
- After submission, show `Correct Answer!` if the selected answer matches; otherwise show `Incorrect Answer`.
- If the answer is incorrect and the answer has been submitted, also show the correct answer.
- Display `Submit Answer` before submission.
- After submission, display `Show Results` if it is the last question; otherwise display `Next Question`.
- You only need to render the appropriate button; button functionality is not required.

Concepts: Combined conditions, nested conditional output, data-driven rendering.

Q14. Search Results with Empty and Error States

Create a component named `SearchResults` using these variables:

```
const isLoading = false;
const hasError = false;
const searchTerm = "react";
```

```
const results = [
  { id: 1, title: "React Basics" },
  { id: 2, title: "Conditional Rendering in React" }
];
```

Requirements:

- If `searchTerm` is empty, display `Enter a search term`.
- If a search is in progress, display `Searching...`.
- If an error occurs, display `Something went wrong`.
- If results exist, display their titles in a list.
- If the search completes successfully but there are no results, display `No results found for "react"`.
- Give these states a sensible priority so contradictory messages are not displayed together.
- Use each result's `id` as the React key.

Concepts: State priority, early returns, empty states, list rendering.

Q15. Conditional UI for a Course Enrollment Card

Create a component named `CourseCard` using the following data:

```
const course = {
  title: "React Development",
  fee: 5000,
  seatsAvailable: 0,
  isEnrolled: false,
  isCompleted: false,
  isPublished: true
};
```

Requirements:

- If the course is not published, display `Course Unavailable` and do not display enrollment details.
- If published and the student has completed it, display `Course Completed`.
- If published, not completed, and already enrolled, display `Continue Learning`.
- If published, not completed, not enrolled, and seats are available, display the fee and an `Enroll Now` button.
- If published, not completed, not enrolled, and no seats are available, display `No Seats Available`.
- Ensure the conditions are ordered so the correct message appears when more than one Boolean value is true.
- Keep the card's JSX readable; use early returns or a helper function if needed.

Concepts: Compound conditions, precedence, early returns, real-world conditional UI.

Recommended Learning Order

1. Start with Q1–Q3 to master the basic syntax.

2. Complete Q4–Q8 to practise multiple conditions and alternative approaches.
3. Complete Q9–Q10 to combine conditional rendering with arrays and attributes.
4. Attempt Q11–Q15 to practise realistic UI scenarios and more complex logic.

Quick Self-Check

Before considering a question complete, check:

- ☐ Does the component render the correct output for every specified condition?
- ☐ Are mutually exclusive states handled in the correct order?
- ☐ Are repeated elements rendered with stable `key` values?
- ☐ Is repeated JSX avoided where a shared structure would work?
- ☐ Are empty, error, and unexpected-value cases handled where required?
- ☐ Is the code readable and properly formatted?